



ProteoML: User Guide

Apply machine learning to predict the post-translational modification activity of an enzyme with the click of a button

Created by Nashira H. Ridgeway

December 2024

Table of Contents

Quick-Start Guide	3
1 – Download Peptide List	6
2 – Upload Peptide Results	8
3 – Fit Models	12
4 – Ensemble Learning	17
5 – Experimental Predictions	20
Troubleshooting	22

Quick-Start Guide

1 – Download a Testable Set

In the first tab (“1 - Download Peptide List”), you’ll find 8 different buttons, each representative of cleaned datasets of 15 amino acid (AA) long peptide sequences, encompassing real sites within proteins known to exhibit the specified post translational modification (PTM). Simply click the button to select a dataset (you may choose more than 1!) or click it again to unselect it. To download your selected dataset(s), click the “Download File(s)” button.

2 – Upload Experimental Results

Once you’ve experimentally determined the activity of your chosen enzyme towards a list of synthesized peptides (such as those downloaded in Step 1), you must define activity as either a 1 or a 0 (1 = activity present, 0 = no activity present). The machine learning models must have binary (1’s and 0’s) indicators to be able to function.

Your input needs to be in table form and must include some sort of identifier for the protein/peptide (Uniprot ID suggested, but anything is accepted), the AA sequence of the peptide, and that binary value discussed previously. With these basics, one can create a basic ML model and apply it to an experimental dataset (i.e. the full proteome).

NOTE: All peptides **must** be the same length. If you have a dataset of peptides of unequal length, see Page 8 for more information on padding.

NOTE: ProteoML works best for peptide sequences ranging from 13 to 15 AAs in length—you can increase the size, if need be, but feature generation will take longer!

NOTE: To create an ensemble ML model, you’ll need a secondary ML score. We suggest using MusiteDeep, or a similar ML predictor that can either score the sites within proteins (ideal, as the full protein is considered), or the peptides themselves for the likelihood that your PTM of interest may occur. The MusiteDeep scores have been precomputed for the preloaded datasets described in part 1 above (titled “SECONDARY_ML_SCORE” in the files).

NOTE: Predicting whether a PTM of choice may occur (e.g. can this site be methylated?) is different from whether your enzyme of interest will enact the PTM (e.g. does SET8 methylate this site?)—that’s what ProteoML is for.

Please see Page 8 for more information on uploading your experimental results.

3 – Generate the Base Machine Learning Model

The easiest way to initiate this step is to click the “Fit Models and Balance Data – Return Best Fit” button at the top of the page. ProteoML will automatically fit all available ML models and sampling types to your data, returning the best-performing combination for your dataset, along with the Precision-Recall and Threshold v Performance Metrics Graphs.

ProteoML will also determine the ideal threshold for positive vs. negative classification (i.e. scores over 0.30 will be deemed as positive, the rest negative), based off model performance. Scrolling down from the graphs, you’ll be able to see the selected threshold, along with “Highly Associated GO Terms” if your training data used human Uniprot IDs. These GO Terms are for molecular function, and the number of training data sites that correspond to a particular term are listed under “go_count” in the table. Feature Importance, or how vital each of ProteoML’s automatically generated features are for the resulting ML model are also listed in the table just beneath the GO Terms.

You may save the graphs, along with the GO Terms and Feature Importance by hitting the “Click to save metric curves and related data” button at the bottom of the page. Be sure to take note of the model and data balancing types ProteoML chose for your data.

Please see Page 12 for more information regarding the Base ML model.

4 – OPTIONAL: Generate the Ensemble Machine Learning Model

If you’ve gotten as far as generating the base ML model, and included Secondary ML scoring within your training dataset, you can proceed with ensemble learning! Much like the base model, we suggest you select the “Fit Models and Balance Data - Return Best Fit” button. This will automatically determine the best ensemble approach to handle your base ML model and secondary ML scores, once again outputting the metric graphs. Be sure to take note of the model and data balancing types ProteoML chose for your data.

Please see Page 17 for more information regarding Ensemble ML models.

5 – Apply the Model to Search for New Substrates

The final stage of ProteoML is applying your newly created ML models to the proteome, to score and determine potentially novel sites of PTM activity induced by your enzyme of choice. To facilitate this, ProteoML comes preloaded with proteome-wide sets for certain PTMs (see the dropdown menu under “Option 1”). Users may upload their own list of peptides following the steps of “Option 2”. After generating the experimental features, a process that can take quite a while depending on the size of your dataset,

we suggest you save that feature file locally so you may come back and re-run with other models or repeat the experiment if required – do so by clicking the “Save ProteoML Computed Features for the Experimental Set” button. If re-uploading precomputed experimental features, follow “Option 3”.

Once your features are computed, select the base ML or ensemble ML model configuration, then the “Initiate Experimental Machine Learning Run” button. This process should not take very long. Once completed, you may save the file locally by hitting the “Save Score File Locally” button.

NOTE: If you did not include secondary ML scores, or did not go through the optional steps on tab 4 to create an ensemble model, you will not be able to select the ensemble ML model for the experimental machine learning run.

Please see Page 20 for more information regarding the application of your models to experimental sets.

Saving Graphs and Associated Files

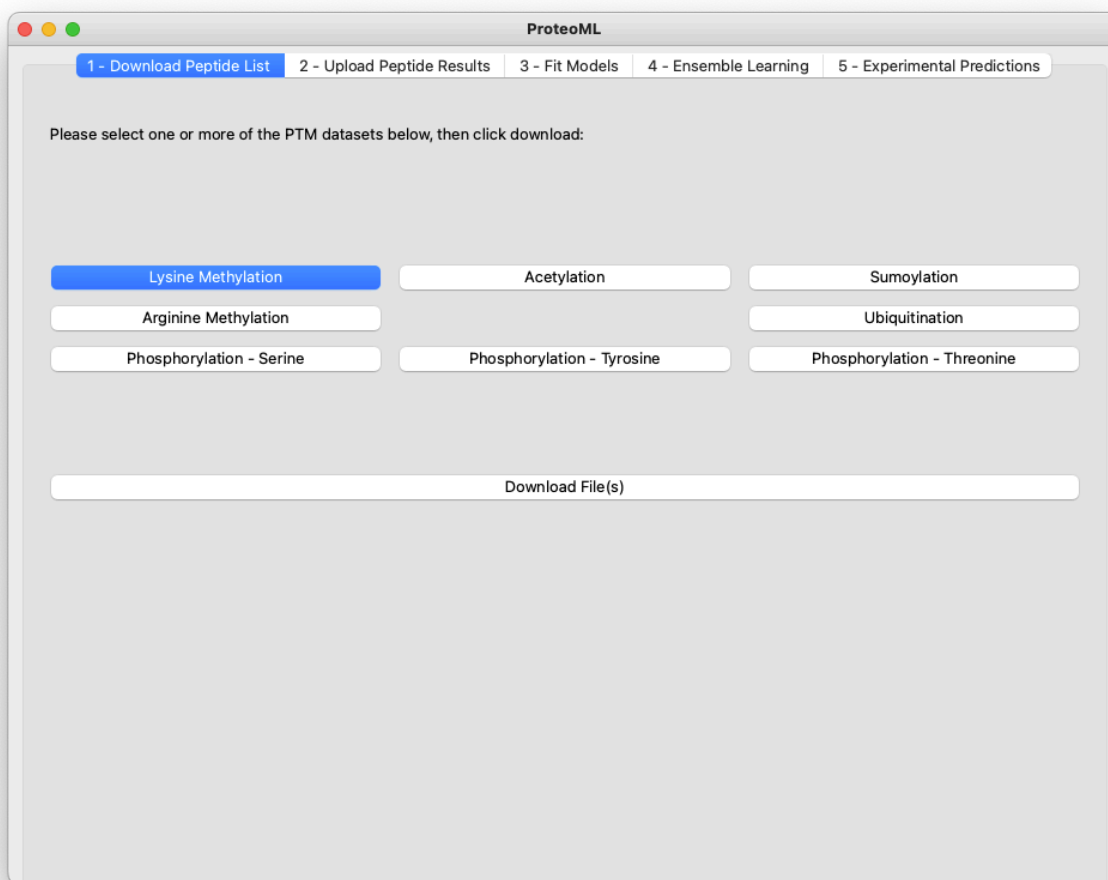
Upon exit, ProteoML will prompt you with a popup message asking if you meant to quit. If you have run any component of the ProteoML process (i.e. feature generation, ML model fitting, experimental scoring, etc.), the popup will include an option to save your files prior to exit. Select the place where you’d like the files saved, and a repository will appear there with the associated graphs, excel files, and logs of the run.

NOTE: If you forgot to take note of the ML models and data balancing approaches that worked best for your data, the logfile should list that information (best model..., best sampling...)

Next, explore the list of new, potential sites your enzyme modifies!

A relevant follow-up investigation may be to isolate the Uniprot IDs of the sites predicted to be positive for further analysis with ProteoML Visualizer. Simply isolate the Uniprot IDs within the ProteoML_predictions... file that have a 1 within the “Final ML Classification” column (this file is automatically sorted so the 1, or positive for predicted PTM activity, sites are at the top).

1 – Download Peptide List



As shown above, the lists preloaded to ProteoML are as follows.

- Lysine Methylation
- Arginine Methylation
- Acetylation
- Sumoylation
- Ubiquitination
- Phosphorylation – Tyrosine
- Phosphorylation – Serine
- Phosphorylation – Threonine

To download a dataset, simply click on the button depicting the dataset related to your enzyme of interest. You can select more than 1. On a click, the button should depress,

or be “selected”, unless clicked once more (to deselect). To download the chosen dataset(s), click the “Download File(s)” button. This will bring up a file saving dialogue where you can specify where the files should go.

Each dataset was downloaded and formatted from PhosphoSitePlus in February of 2024. Formatting included ensuring the sequences replaced multivariate (i.e. X or blank) amino acid values with alanine, as it has no sidechain and should cause relatively little effect to peptide based experiments. Columns were included to be modified by the user, such as the “EXPERIMENTALLY_ACTIVE” and “SECONDARY_ML_SCORE” columns.

As discussed in the Quick Start guide on Page 3, the experimentally active column requires PTM activity for the assessed peptides to be represented by binary values, meaning 1 for activity, or 0 for no activity. The secondary ML score values are optional, but likely will improve your model’s capability in identifying novel sites of activity when applied to the proteome or experimental set. See the next chapter for additional information on formatting peptide lists for ProteoML.

2 – Upload Peptide Results

To ensure ProteoML functions properly, it is very important that your experimental data is formatted correctly. The easiest way to ensure proper formatting is to ensure at least three of the six dropdown menus listed on Tab 2 of ProteoML may be found as columns within your data file. A breakdown of each dropdown menu found on Tab 2 is below.

Protein ID (required): An identifier for your peptide sequence. Ideally, the Uniprot ID for the protein which your peptide was obtained from.

NOTE: Your Protein IDs should be unique, though it isn't necessary.

NOTE: Including Uniprot IDs rather than other formats enable the creation of the GO Molecular Function Annotation table shown in Tab 3 of ProteoML, providing additional insight into the experimental data you're using to train the model with. This table gets updated as soon as feature generation is completed (meaning it's available to you prior to any ML fitting, assuming you used and indicated human Uniprot IDs).

Site Location: Ideally, these values should represent the number at which the central, potentially modified AA falls within the full protein sequence. ProteoML works with peptides, and these peptides don't need to be taken from full protein sequences, but the program works best when reflecting what occurs within biology. Hence, creating peptide sequences from sites that may be modified within proteins best reflects the biological landscape. Moreover, maintaining this strict type of notation ensures you'll have clean data come publication (fingers crossed!).

NOTE: Site Location is not required by ProteoML, only suggested for best-practices in terms of biological context.

AA Sequence (required): This column contains the peptide sequences that were synthesized and tested for PTM activity by your enzyme of choice. **It is imperative that each sequence in this column is the same length, or else ProteoML cannot function.** ProteoML has been tested with peptide sequences from 13 AAs to 25 AAs in length. The main effect of longer peptide sequences is the time it takes to generate features. It is suggested that peptide sequences remain below 35 AAs at most.

NOTE: If you have sequences of varying lengths and cannot retest them at a uniform length, you can pad either end of the sequence with alanine (A). Be sure your central AA remains in the middle of the resulting sequence though! See below for an example.

`_ _ G R A K P Y Q T _` $\Rightarrow$ `A A G R A K P Y Q T A`

NOTE: Peptide sequences must be written in the single-letter amino acid code format, and undetermined amino acids must be written with a 'X' (they will be replaced with

alanine, or A by ProteoML automatically). If you have characters that don't conform, an error will be thrown.

Experimental (required): This column has the binary value for PTM activity. If your PTM showed activity for a peptide tested, represent the experimental value for that peptide with 1. For no activity, use a 0. It's up to the user to determine the cutoff for PTM activity, however in testing any spot that demonstrated a signal above 2 standard deviations of the mean background value (i.e. the background signal of the peptide array), were deemed as positive.

Secondary Score: To generate the ensemble ML model, you'll require a secondary ML score to combine with the output of your base ML model. For lysine methylation, we suggest MethylSight, but for more generalized PTM sites, we applied [MusiteDeep as it was readily accessible via a webserver](#). We encourage users to employ secondary ML predictors they have experience with or easy access to.

Secondary ML Validation Set: The values in this column must be one of two words/numbers/letters, indicating whether the peptide in that row is a part of the secondary ML holdout set or not. **Only include this if you've received a defined holdout set from the creator of the Secondary ML Model you're using within your dataset, that you were able to test for PTM activity experimentally.** In testing, we found that ProteoML runs well without this value—however this is a “gold standard” approach that should always be applied if available. If no holdout set is defined within the dataset uploaded, ProteoML reserves 30% of data for validation (the other 75% is applied to train the model, with cross validation techniques).

NOTE: A holdout or validation set is a dedicated portion of your data withheld from the training process of the ML models. Once finalized, the validation/holdout set is applied to properly gauge the performance of a model, as these data have never been seen by your model during training. In ensemble learning, when you apply two ML models, defining a proper holdout set can get tricky, especially when you are only creating one of the models yourself (as in our case here). Sometimes, if you're lucky enough to be dealing with some very kind authors, they may define the holdout set they applied to test their model within their published data. This is the case with MethylSight, though unfortunately not for MusiteDeep. In the case of MethylSight, the Secondary ML Validation set was identified by peptides having a “MethylSight_Holdout” in the column, as opposed to nothing for the peptides that weren't a part of the set.

ProteoML accepts excel (.xls, .xlsx), .csv, or .txt files as input on this tab. **Any column titles are acceptable and get automatically detected by ProteoML** once you click “Upload Result Dataset”. A sample file is shown below.

Uniprot ID	PTM Position	Peptide	Experimental Value (1 = Y)	Secondary ML Score	Secondary ML Holdout
Q9Y7Q6	145	DSEGLKTVPIRE	1	0.75	YES
Q29537	20	ICTIDVKMIMVKA	0	0.012	NO
Q8Y6Q4	214	GRYYEGKYAAGPG	1	0.32	NO

To inform ProteoML as to which column contains what, select the appropriate column name in the dropdown list corresponding to the six categories listed above. Once

completed, click “Initiate Feature Generation” to have ProteoML automatically generate the features for you. ProteoML applies three types of features to your peptide data, for it to be numerically represented so the ML models can be fit to it. These are as follows:

- 1. Protein-Related Information:** Measurements such as Levitt’s Protein Conformation Probability, molecular weight, and more, are calculated based off the amino acids that make up the peptide. For more information, we point the reader towards [ProtDCal’s approach to structural representation for ML](#).
- 2. Sequential Information via one-hot encoding:** Systematically encoding the identity and position of each AA that composes the peptide is completed through one-hot encoding. Essentially, we end up with positional columns (i.e. lysine at position 3), where a 1 indicates the presence of that amino acid within the peptide sequence.
- 3. Molecular Structure via MACCS Keys:** Typically applied in ML spaces to investigate how a drug will interact with a protein, Molecular ACCession System (MACCS) Keys encode the molecular structure of a protein through 166 different keys, each representing a different molecular element (i.e. C=C bond).

These features are also applied to any experimental set you upload in Tab 5, so the ML models may accurately “see” the new peptide sequences so they can then classify them as predicted to gather a PTM from your enzyme, or not. Feature generation can take a long time, but fear not! Be sure your computer’s connected to a power supply and awake, and the process can run in the background.

The feature generation process will automatically detect and remove duplicate sequences, as the presence of duplicates will negatively affect model performance. Additionally, once feature generation is completed, ProteoML will output the count of positive and negative data detected within your dataset. Datasets that present fewer than 20% of the positive or negative case will be deemed as imbalanced, and future automated processes will take this into account.

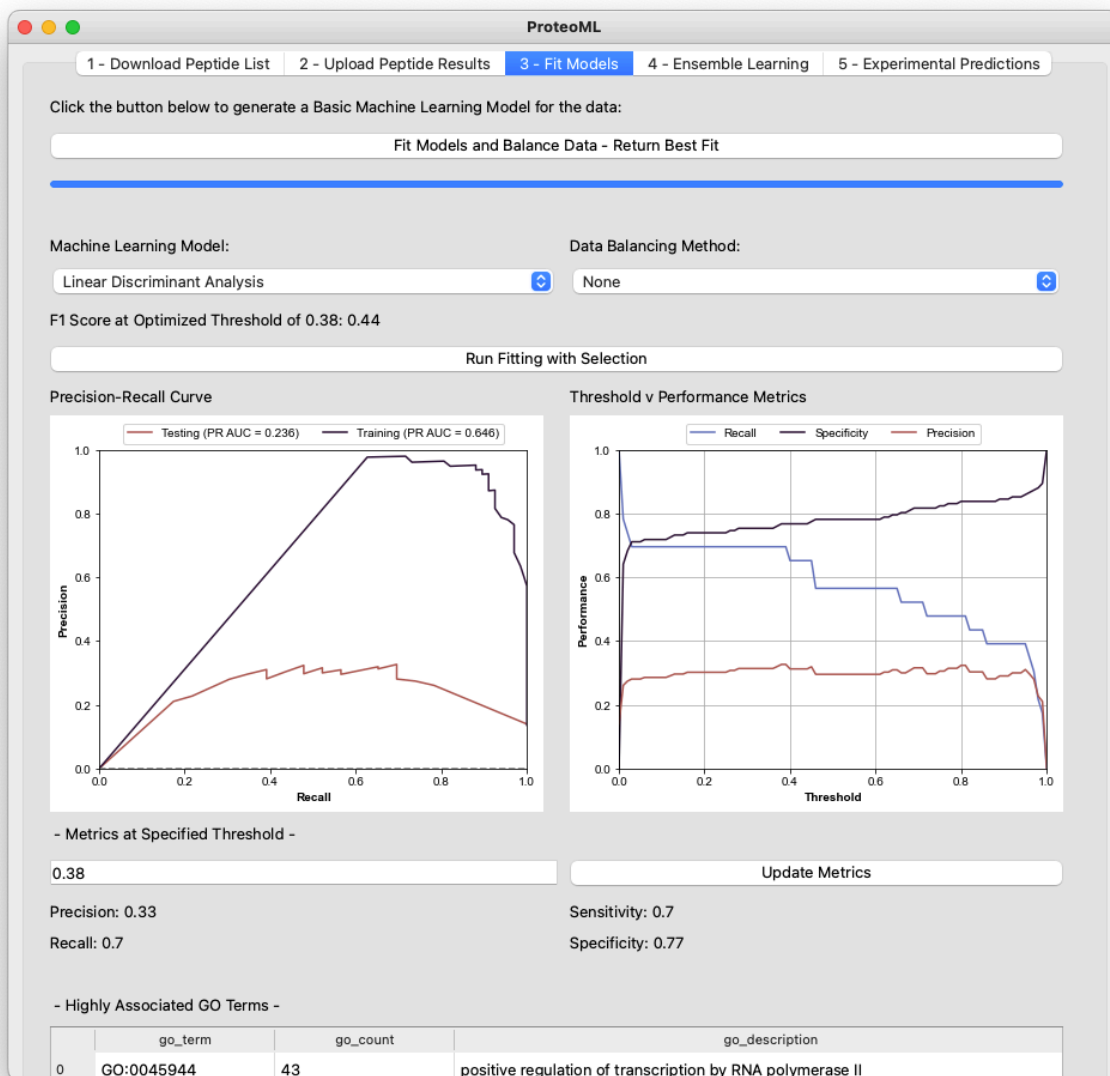
Considering the time, it can take to generate features for a lengthy dataset, ProteoML has the option to save computed features using the “Save ProteoML Computed Features” button. You can reupload those features in this same tab (Tab 2), scrolling down to “Option 2: Upload Previously Computed ProteoML Feature Set”. Click that button and ProteoML should automatically recognize the features you’ve computed (NOTE: these files can be pretty big!). **Just be sure not to alter the feature set once it’s been saved—that’ll affect if ProteoML can read it back in!**

3 – Fit Models

Well done, reader. You’ve done all your wet-lab peptide experiments, cleaned up your data, generated your features, and now, *finally*, we can get to the machine learning!

As mentioned in the quick start guide, unless you’ve somewhat familiarized yourself with ML models and data balancing/sampling approaches, your best bet is to click the “Fit Models and Balance Data – Return Best Fit” button and move on with your life. If you’ve done so, you’ll have to wait a little while for the process to run, but will eventually be greeted by two graphs, some values, followed by two tables.

Precision-Recall: The first graph is the Precision-Recall curve, in which precision, a measure represented by the amount of correctly classified, true PTM-modified sites determined by your ML Model divided by all the sites classified as true (correctly or incorrectly classified by your ML Model as they may be $\text{precision} = [\text{true positives}] / ([\text{true positives}] + [\text{false positives}])$). Recall is also defined by the amount of correctly classified, true PTM-modified sites determined by your ML model, however that value is divided by all the sites that are observably true within your experimental set (i.e. experimentally deemed as positive for PTM activity; $\text{recall} = [\text{true positive}] / ([\text{true positive}] + [\text{false negative}])$). Ideally, this curve aligns to the upper right quadrant of your graph—the closer it is, the better, though often in practice, the curve falls a little closer to the middle, or the very bottom of the x-axis. You’ll note that there are two lines presented on the graph—one for the training data, and another for the testing. This helps to ensure that your model is not overfit.



An example of an overfit precision-recall curve. Note that the training AUC of 0.646 is well over 25% larger than the AUC of the testing set (0.236).

A note on overfitting: If you're concerned about overfitting, look at the generated Precision-Recall Curve. **Is the training line much better (i.e. closer to the upper right-quadrant of the graph), and leagues above the testing line? Does it look like the picture above? Then your model is overfit (bad!).** An overfit model means your model is excellent at identifying positive and negative examples in your training data, so good in fact, that when it's applied to a never-seen dataset it performs terribly. This spells trouble for when you apply it to the proteome in search of new sites. Try a more basic model, such as "Linear Regression", and do not apply any data balancing to see if this graph improves.

Class Imbalance: If your dataset has an imbalance of 80:20 or greater towards the positive or negative case, ProteoML will employ F1 or F-score scoring to determine the fit of your model. The F-score is the harmonic mean of precision and recall ($F = 2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$). Otherwise, ProteoML uses the area under the curve (AUC) of the ROC curve.

Threshold v Performance Metrics: Once the base model has been fit, ProteoML will automatically display a graph displaying Threshold v Performance Metrics. Three performance metrics: Recall, Specificity, and Precision, are displayed upon the curve, at various thresholds on the x-axis. The thresholds are different cutoffs for determining the positive or negative classification of a peptide, based on the ML score generated. As an example, a threshold of 0.5 is typically applied to ML classifiers. In this case, data that scores above 0.5 will be deemed as positive, whereas anything falling below 0.5 is negative. When your data has a class imbalance, like that frequently analyzed in ProteoML, it's good practice to determine the effect that changing the ML classification threshold will have on performance. **If your dataset exhibits an 80:20 or more class imbalance (i.e. 90:10), ProteoML will automatically perform threshold analysis to determine the threshold that results in the best F-score and display it in the Metrics at Specified Threshold section** (to be discussed further below). Going back to the Threshold v Performance Metrics graph, once you've decided on a threshold, it's good practice to indicate it manually on the graph, especially when publishing or presenting the visual as it demonstrates model performance and optimization well.

ROC Curve: A Receiver Operating Characteristic (ROC) curve is also generated for both the base and ensemble models. This curve is not included within the main ProteoML console, as it will give a false indication of performance, unless data is balanced, which often is not the case in ProteoML. ROC curves measure the rates of true and false positive identification. Although true positive rate ($[\text{true positives}] / [\text{true positives}] + [\text{false negatives}]$), indicates how well our model is identifying actual positive sites of PTM activity, the false positive rate ($[\text{false positives}] / [\text{false positives}] + [\text{true negatives}]$) will be falsely inflated, as often the peptide library searches required by ProteoML will return a high proportion of negative data. Exposing all this negative data to the model makes it very good at identifying true negatives within a new dataset, however it will be bad at identifying true positives within a new dataset, as it has seen very little positive data. **Using a metric that weighs negative and positive classification rate so equally does not work for our purposes, as our models will always be worse at identifying positive data points (as the model's only seen a handful of examples). Hence, metrics that consider true positive rate, such as precision and recall as described above, are much better applied to ProteoML.**

NOTE: If your dataset is imbalance is more equal than that of the 80:20 split, save your metric curves and report the ROC curve instead of precision-recall. ProteoML will be

employing the AUC ROC metric to assess fit, so it's best to keep form by using the ROC curve rather than precision-recall for main model assessment. It is, however, always good to have a precision-recall curve, and the Threshold v Performance Metrics graph is another useful resource.

Metrics at Specified Threshold: As mentioned above, if your dataset is imbalanced by more than 80:20 (i.e. 95:5), the F-score will be used to determine the best classification threshold for your model. This threshold is printed out in the modifiable box, just above various ML metrics for your base ML model. Precision and recall are included, along with sensitivity ($[\text{true positives}] / ([\text{true positives}] + [\text{false negatives}])$) and specificity ($[\text{true negatives}] / ([\text{true negatives}] + [\text{false positives}])$).

Changing the Threshold: You can modify the threshold value in the box just below the header of this section but be sure the value selected is a decimal that falls between 0 and 1. We strongly suggest not simply “picking” a threshold as the chosen threshold is based off of statistical analysis, and selecting one that you think is better, is not reflective of the data or your model. If you want to avoid using the threshold all together, we suggest carrying on until Tab 5 to apply the ML model to your dataset, generating ML scores for your base (and potentially ensemble) ML models, and selecting the top n scoring peptides or protein sites.

Highly Associated GO Terms: If you used human Uniprot IDs within your feature set, upon feature generation, this table was filled out and displayed. Gene Ontology (GO) terms, descriptive of molecular function, are assigned to the Uniprot IDs from a database retrieved from the [Gene Ontology Resource](#).

Important Features: Feature importance defines the dependence of our model on each of the features generated by ProteoML (see Section 2 for more information on the types of features applied by ProteoML). The importance of each feature is calculated, and output in the Important Features table. A graph of important features (in both the positive and negative case, if applicable) is output when you click the “Click to save metric curves and related data” at the bottom of Tab 3.

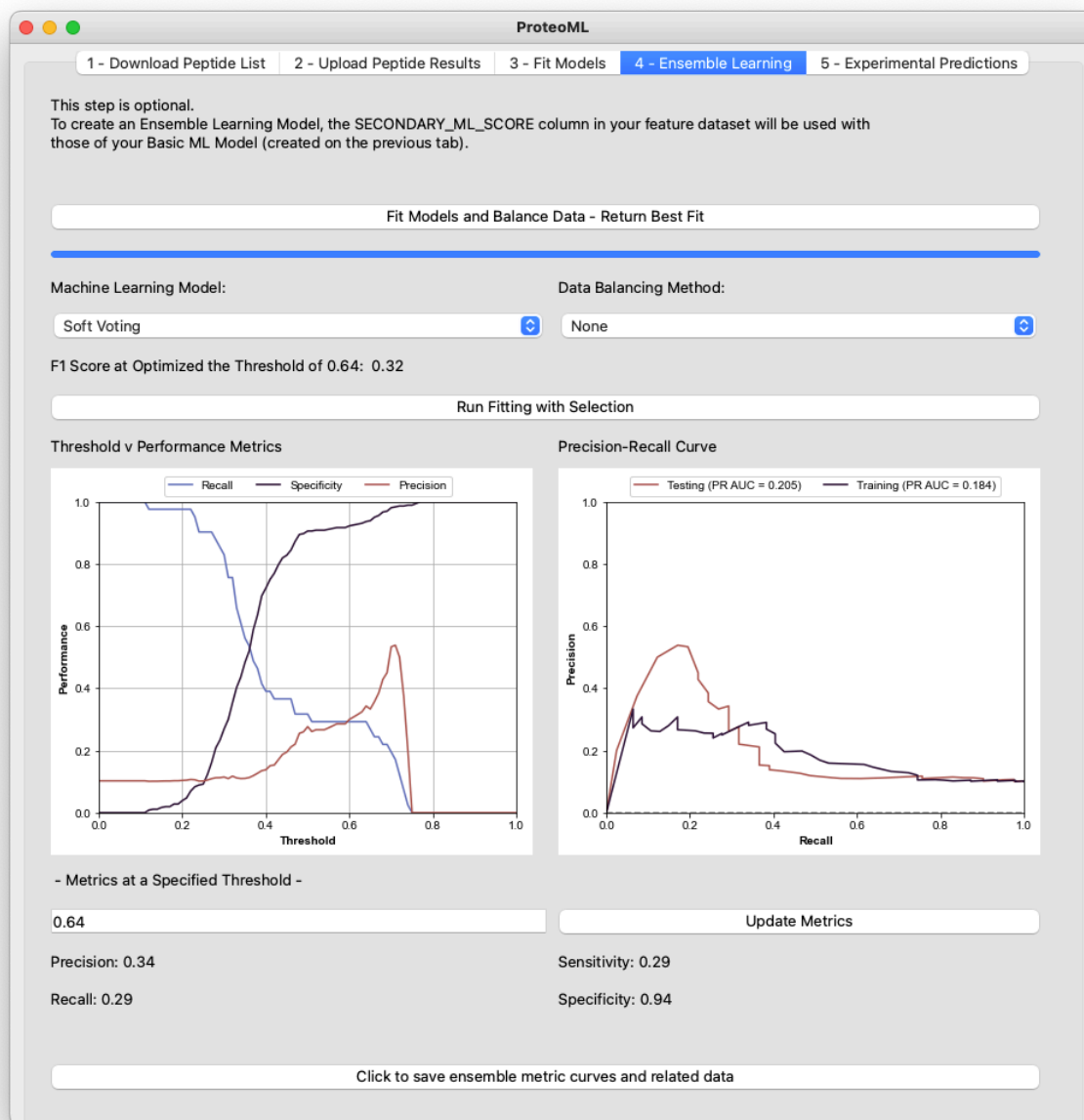
ML Model Types: A total of 11 different model types are available for the base ML model. The Dummy Classifier applies no logic, only randomly selecting members of the positive and negative class. It's included for testing purposes, and as a possible comparison between a “coinflip” (or totally random) model, with a good model. **The models are ordered from the simplest to the most complex, starting with Logistic Regression (the simplest), and ending with Gradient Boosting (the most complex).** If you have concerns of overfitting, it's best to go with a simpler model, and vice-versa for underfitting. A great beginner-friendly resource to decode the difference between these models is [StatQuest with Josh Starmer on YouTube](#).

Data Balancing: There are 14 different data balancing approaches available for the base ML model. If you have serious concerns about overfitting, we suggest selecting “None”, and simply proceeding with the data balance as-is. Otherwise, you may want to apply oversampling methods, in which the underrepresented class is sampled more based on differing strategies, or undersampling methods which revise the dataset to delete some data of the overrepresented class. The simplest data balancing methods are the Random Oversampler or Random Undersampler, which as the names suggest, randomly over or undersample the under or overrepresented classes respectively. Combined data balancing methods utilize both over and undersampling in creative approaches. If inclined, we suggest the user researches the data balancing methods individually as some are based on complex mathematical concepts.

Saving your progress: We strongly encourage users to save their files along the way, in the odd chance that the program could exit unexpectedly. The “Click to save metric curves and related data” button will do just as it says, saving all metric curves (ROC, precision-recall, performance metric v threshold, feature importance), as well as associated tables (highly associated GO terms, important features, etc.). **It is vital that the user take note of the machine learning, and data balancing approaches applied, as those will be of use come publication.**

4 – Ensemble Learning

This step is optional but recommended. In our findings, we observed that the ensemble approach enables the final ML model to perform better within the proteome. That means you'll be more likely to find high-quality sites of PTM activity for your enzyme.



As mentioned, the first step in ensemble learning is predicting with a Secondary ML Model. MusiteDeep is a great place to generate scores for your experimentally validated peptides, and the proteome-wide search, if you decide to do so. Ideally, you'd apply the Secondary ML Model to the full-length protein sequences your peptides are taken from, but in practice that often requires cleaning and formatting data, as well as lengthier

runs. If you can make it happen, we suggest you do so, otherwise completing the run with just peptide sequences is perfectly acceptable. **Regardless, to run ensemble learning, you'll need to annotate both the peptide results file you upload in Tab 2, as well as the experimental predictions file you upload later in Tab 5, with the Secondary ML Scores.** Be certain you inform ProteoML of the location of the Secondary ML Scores in the appropriate dropdown menus within Tabs 2 and 5.

NOTE: For your convenience, the experimental proteomes uploaded to ProteoML come pre-annotated with MusiteDeep scoring, or MethylSight scoring in the case of lysine methylation, as defined in the dropdown menu in Option 1 on Tab 5.

As in the base ML model fitting achieved in Tab 3, users may select to automatically determine the best fit for their ensemble model. This process takes less time, as there are less features, and the models automatically applied are more simplistic. To prevent overfitting, the approaches employed are Soft Voting, Logistic Regression, and Linear Discriminant Analysis only. No data balancing approaches are employed by the autofitting process, as testing found that to increase the chances of overfitting.

See below for instances when you should select for the model/data balancing approach yourself.

A note on overfitting: See the corresponding section on Page 13. If you think your ensemble model may be overfit, try a more basic ensemble model, such as “Soft Voting”, and do not apply any data balancing to see if this graph improves.

A note on underfitting: Users have the option to select a more complex ML model and apply data balancing, though that isn't suggested, unless you're seeing underfitting in your model. **Looking at the precision-recall curve of the ensemble ML model, if the training and testing lines are both close to the y-axis and very close together, your model may be underfit to the data.** This means it underrepresents the data and will barely find any potential PTM sites within the proteome. It may benefit from a more complex ensemble approach. Try a few to see which option produces a good graph.

Getting things just right: The ideal precision-recall curve has the training data performing better (i.e. closer to the upper-right quadrant of the graph), than the testing data. But not by an obscene amount, typically by 25% or under. Typically, if the training and testing lines are somewhat mimicking each other, but have a bit of a gap between them, your model is fit well. It's important to remember that things will rarely look perfect, especially since the data we use to train these models is so finite.

Machine Learning Model: A similar list of ML models applies here as in section 3, with the novel addition of “Soft Voting”. In soft voting, the weighted means of the base and secondary ML scores are applied. We strongly suggest you keep your ensemble ML models simple, with Soft Voting, Logistic Regression, or Linear Discriminant Analysis—

this is the best way to prevent overfitting. See the sections above for additional information.

Data Balancing Method: The same list of data balancing approaches applies here as in section 3. We strongly advise against the use of data balancing approaches (ProteoML will not automatically apply these when running “Fit Models and Balance Data – Return Best Fit”), as they will make your ensemble model prone to overfitting.

Threshold: As mentioned in section 3, ProteoML will automatically determine the best threshold for your model. This threshold will determine what your model classifies as positive or negative for a PTM when faced with new sites that may be affected by your enzyme of choice. A higher threshold means less sites will be deemed as positive and may improve the precision of your model. However, a lower threshold may prevent you from losing out on potential sites that weren’t scored so high by your model. This careful balance is automatically determined for you by ProteoML, but you can alter the threshold if you want a more stringent or permissive search. You can see how the threshold affects your model’s performance through the Threshold v Performance Metrics curve, as well as changing the number to any decimal between 0 and 1 in the box beside the “Update Metrics” button (click that to apply your new metric). The Sensitivity, Specificity, Precision, and Recall metrics will update, and this new threshold will be applied in Tab 5 to determine a positive or negative classification for your data.

Take note of the threshold ProteoML determines for your model—you can always change it back!

Saving your progress: Click the “Click to save ensemble metric curves and related data” to output a file containing the .pdfs of your precision-recall, ROC, and threshold v performance metric curves, along with an excel file containing the metrics used to generate those curves at 100 different thresholds for your ML model.

5 – Experimental Predictions

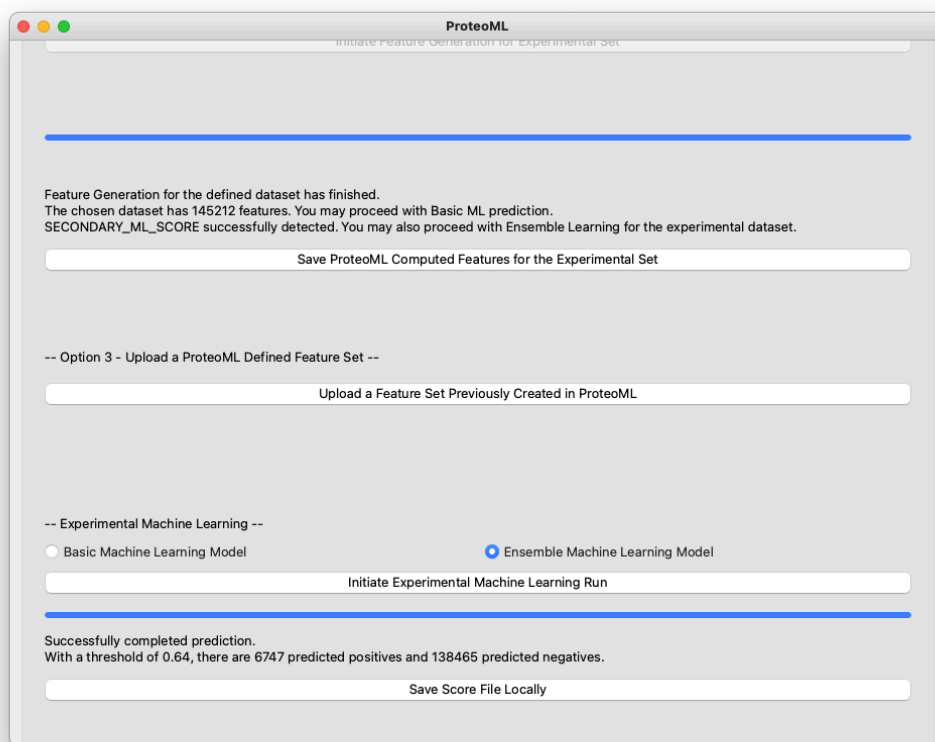
After all the feature generation, model fitting and assessment, you can finally test an experimental set with your model. Bad news however—you may have to do some more feature generation first (sorry!). You have three options to consider in terms of experimental sets to apply your ML model to.

Option 1 – Select a Dataset: This option is the easiest. If you're able to find a dataset in the dropdown menu that suits the enzyme you're investigating and resembles the training data you used to create your first model on Tab 3, use it! These datasets have already been scored with MusiteDeep for convenience, but if that isn't what was applied in your model, feel free to download the dataset and modify the scores yourself. Unless selecting the "Refined Surface Exposed Sites" MethylSight option, ProteoML will need to compute the features for your dataset. This'll be automatically initiated once you hit the "Proceed with Selected Dataset" button.

NOTE: If you did not initialize, or do not want to use the ensemble ML model, simply select the "Base ML Model" option beside the "Initiate Experimental Machine Learning Run" button. This will be automatically selected for you regardless of if you initialized the ensemble ML model or not (and the ensemble ML model option will not be selectable if it was not created). When scoring your experimental set, ProteoML will only use your base ML model.

Option 2 – Upload a Dataset: This option is best for those with a unique dataset they'd like to analyze, such as a subset of the proteome, or to score a mass spectrometry generated list of peptides. As with the feature set in Tab 2, if you want to apply ensemble learning, you'll need to include a column of Secondary ML scores produced by MusiteDeep or another generalized predictor for your PTM. Click the "Upload a Dataset Containing Peptide Sequences" to upload your file (.xls, .xlsx, .csv, .tsv accepted), and your column names will be automatically included in the four dropdown menus. Like Tab 2, indicate the column of Protein IDs, as well as the Site Location, AA Sequence, and Secondary Score (see Page 8 for more detail). Once indicated, select "Initiate Feature Generation for Experimental Set" to run the feature generation. Once again, you can save your features for future ProteoML runs by clicking the "Save ProteoML Computed Features for the Experimental Set" button (warning – these files will be large!).

Option 3 – Upload a ProteoML Defined Feature Set: If you ran the feature generation for a large experimental set before, select the "Upload a Feature Set Previously Created in ProteoML" button to reinitialize it in this session.



Initiating the ML Run: Select the model you wish to apply to your dataset (Basic or Ensemble), then click the “Initiate Experimental Machine Learning Run” button. This process shouldn’t take long. ProteoML will output the count of positives and negatives for your dataset based off of your chosen model.

NOTE: You cannot select the Ensemble Machine Learning Model to run your data on unless it’s been created in previous tabs, meaning you need to include a Secondary ML Score in your initial dataset, and experimental dataset.

NOTE: If you do end up scoring the experimental dataset with an ensemble model, as the base ML scores are a key component to that, they’ll also be listed in your save file of experimental scores.

Saving the Scored Dataset: Click the “Save Score File Locally” to initiate a file saving popup. Or alternatively, click the ‘x’ to exit the application, and a popup will be shown asking if you want to save your data for the run. Select ‘Yes’ and indicate which file you want the repository to be saved to. ProteoML will automatically detect what you haven’t already saved from your run, and include it there.

Troubleshooting

Possible Error/Warning Messages

Secondary Score validation set not specified. Verifying the performance of the ensemble model is always best with a validation set, but it is not necessary to carry forward in model fitting and predictions.

A predefined holdout/validation set for the Secondary ML Model was not defined in the feature columns. It's the best practice to supply one, though they're rarely available. This is simply a warning (see Page 9).

Uploaded feature file, but it contains improper columns. If you have edited the feature file, please revert to the output file and try again.

The precomputed file of features was uploaded, but the columns did not match up with what ProteoML requires. Perhaps be sure you're re-uploading a ProteoML generated feature set (i.e. has headers such as "ONE-HOT..."), and not a set of peptides with scores. If you are uploading the latter, it is a dataset—if it has experimentally generated values for PTM activity it belongs on Tab 2 (click Upload Result Dataset under Option 1), otherwise it is likely an experimental dataset and should be uploaded on Tab 5 (under Option 2, click Upload a Dataset Containing Peptide Sequences).

Unable to upload file. Please attempt with a .xlsx, .csv, or .tsv file.

ProteoML was unable to recognise your file. Ensure that your file is one of the types listed in the error message, and perhaps include that corresponding extension in your filename directly.

Cannot proceed with Ensemble Learning as the SECONDARY_ML_SCORE column in the feature dataset doesn't contain any scores. To proceed, populate the SECONDARY_ML_SCORE column with numeric scores from another computational method.

No Secondary ML Score was indicated within your file. Hence, ensemble learning may not be attempted. You may proceed with the base model alone or generate Secondary ML scores and try again.

Error - AA Sequence needs to be short strings of peptides. Reselect or reformat and try again.

ProteoML detected non-letters in the peptide sequences you uploaded. Double check to be sure no spaces, special characters, or numbers and included in your peptide sequence and try again. For more information on proper peptide formatting, see Page 8.

Error - Experimental needs to be binary values of 1 or 0 indicating the presence of activity. Reselect or reformat and try again.

The experimental activity column selected from your uploaded dataset did not contain

just 1's and 0's. Be sure only 1 (positive for PTM activity) or 0 (negative for PTM activity) are included in the file and try again. See Page 9 for more detail.

[Error - Incorrect data type found within the protein/peptide sequence. Please be sure the correct column is selected and try again.](#)

ProteoML detected non-letters in the peptide sequences you uploaded. Double check to be sure no spaces, special characters, or numbers are included in your peptide sequence and try again. For more information on proper peptide formatting, see Page 8.

Common Issues or Concerns

[My Precision-Recall and my Metric v Threshold curves are comprised of mostly horizontal lines.](#)

Double check to see if the ML model applied is a Tree Classifier (i.e. Decision Tree, Random Forest, or Extra Trees Classifier). These ML models work by compiling a series of yes/no questions to sort out the positive data from the negative. The nature of these models cause these horizontal lines, as scoring is more black and white, as opposed to the sliding scale type scoring seen in the other possible models.

[The automatically generated Specified Threshold is below 0.5.](#)

A threshold below 0.5 is common in more restrictive ML classifiers. Often, this can mean that your model hasn't seen enough of the positive case within the training dataset (this is common if your dataset is small—2,000 peptides or less). You'll find your model will generally output low scores, even for positive testing examples. ProteoML corrects for this possibility during the threshold picking—it selects a low threshold for models that score most sites as low. Only modify the threshold if you absolutely must do so. More information on thresholds may be found on Pages 14-15 and 19.

[I ran my data through ProteoML more than once and got different graphs and/or models each time.](#)

Although we attempted to correct for the innate randomness in ML, the fundamental nature of some of the data balancing and ML models is to apply a random state. This causes some deviations to be observed between runs. Don't fret! If you've run things twice and gotten very different results, try running them a third time and take the most commonly applied models and balancing approaches. Regardless of randomness, the data informs which models and balancing types are best suited for it.

If all else fails, we suggest users use the “Issues” section of the ProteoML Github page (<https://github.com/nashirag/ProteoML/issues>) to submit their issues so they may be addressed accordingly.

NOTE: When submitting an issue, it’d be helpful if you could access the logfile within the system files, though it’s not absolutely required. A detailed explanation of how the error occurred (what process failed, what error message displayed if any, etc.) would be extremely useful. Additionally, sending along any screenshots or data files associated with the error would be much appreciated.